



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №9

Технології розробки програмного забезпечення
«Різні види взаємодії додатків: Client-Server, Peer-toPeer, Service-Oriented
Architecture»

Виконав:
студент групи ІА-24
Іскандаров Е. Е. огли
Перевірив:
Мягкий М. Ю.

Київ 2024

Зміст

Короткі теоретичні відомості	3
Хід роботи	4
Реалізація шаблону проєктування для майбутньої системи	5
Зображення структури архітектури	8
Посилання на репозиторій.....	8
Висновок.....	8

Тема: «Різні види взаємодії додатків: Client-Server, Peer-toPeer, Service-Oriented Architecture»

Короткі теоретичні відомості

У сучасному світі розробки програмного забезпечення існують різні підходи до організації взаємодії між додатками, і кожен з них має свої особливості та сфери застосування.

Клієнт-серверна архітектура є однією з найпоширеніших моделей взаємодії. У цій моделі чітко розділені ролі між постачальниками послуг (серверами) та їх споживачами (клієнтами). Сервер виступає централізованим вузлом, який надає певні сервіси та ресурси, в той час як клієнти звертаються до нього із запитом на отримання цих ресурсів. Така архітектура забезпечує ефективне централізоване управління даними та ресурсами, добре масштабується на серверній стороні. Типовими прикладами використання клієнт-серверної архітектури є веб-сайти, системи управління базами даних та сервіси електронної пошти.

Інший підхід представляє однорангова архітектура (Peer-to-Peer або P2P). На відміну від клієнт-серверної моделі, в P2P-мережах всі учасники мають рівні права та можливості, виступаючи одночасно як клієнтами, так і серверами. Відсутність центрального сервера робить такі системи більш стійкими до відмов, оскільки вихід з ладу одного вузла не призводить до припинення роботи всієї системи. Дані в такій мережі розподілені між усіма учасниками, що забезпечує високу надійність зберігання. Найбільш відомими прикладами P2P-систем є файлообмінні мережі на кшталт BitTorrent та деякі месенджери.

Сервіс-орієнтована архітектура (SOA) представляє собою більш сучасний підхід до побудови розподілених систем. Вона базується на використанні незалежних, слабо пов'язаних між собою сервісів, кожен з яких відповідає за певну бізнес-функціональність. Сервіси взаємодіють між собою через стандартизовані інтерфейси, що забезпечує їх взаємозамінність та можливість повторного використання. Це робить систему надзвичайно гнучкою, дозволяючи легко змінювати та оновлювати окремі компоненти без впливу на роботу інших частин. SOA широко використовується в корпоративних системах та при розробці веб-сервісів.

Вибір конкретної архітектури залежить від багатьох факторів, включаючи вимоги до системи, очікуване навантаження, необхідний рівень надійності та масштабованості. Клієнт-серверна архітектура добре підходить для систем з централізованим управлінням даними, P2P ефективна для розподіленого

зберігання та обміну даними, а SOA найкраще проявляє себе в складних корпоративних системах, де важлива гнучкість та можливість повторного використання компонентів.

Хід роботи

Основні принципи P2P архітектури:

Основні принципи P2P архітектури:

1. Децентралізація - відсутність центрального сервера управління. Кожен вузол мережі (peer) може виступати як в ролі клієнта, так і в ролі сервера.
2. Рівноправність учасників - всі вузли мають однакові права та можливості в мережі, можуть як надавати ресурси іншим учасникам, так і споживати їх.
3. Розподілене зберігання даних - інформація розподіляється між усіма учасниками мережі, що підвищує надійність системи та стійкість до відмов.
4. Масштабованість - система може легко розширюватися за рахунок додавання нових вузлів без необхідності значних змін в архітектурі.
5. Автономність вузлів - кожен вузол може самостійно приймати рішення про участь у мережі та надання своїх ресурсів.
6. Стійкість до відмов - вихід з ладу окремих вузлів не призводить до припинення роботи всієї системи завдяки дублюванню даних та функцій.
7. Самоорганізація - мережа здатна автоматично перебудовуватися при додаванні або видаленні вузлів, оптимізуючи маршрути передачі даних.
8. Пряма взаємодія - вузли можуть безпосередньо обмінюватися даними між собою без посередників.
9. Ресурсний внесок - кожен учасник мережі надає частину своїх обчислювальних ресурсів (процесорний час, пам'ять, пропускну здатність) для загального користування.
10. Динамічна маршрутизація - здатність системи знаходити оптимальні шляхи передачі даних між вузлами в умовах постійної зміни структури мережі.

```

1 package org.example.demo2.p2p;
2
3 import java.io.*;
4 import java.net.*;
5 import java.util.Scanner;
6
7 public class P2p {
8
9     3 usages
10    private static final int PORT = 12345;
11
12    public static void main(String[] args) throws IOException {
13        Scanner scanner = new Scanner(System.in);
14
15        new Thread(() -> startServer()).start();
16
17        try { Thread.sleep( millis: 500); } catch (InterruptedException ignored) {}
18
19        System.out.print("Connect to peer? (yes/no): ");
20        String answer = scanner.nextLine();
21
22        if (answer.equalsIgnoreCase( anotherString: "yes")) {
23            System.out.print("Enter peer IP (e.g., localhost): ");
24            String ip = scanner.nextLine();
25
26            System.out.print("Enter message to send: ");
27            String message = scanner.nextLine();
28
29            sendMessageToPeer(ip, PORT, message);
30        }
31
32    1 usage
33    private static void startServer() {
34        try (ServerSocket server = new ServerSocket(PORT)) {
35            System.out.println("Server running on port " + PORT);
36
37            while (true) {
38                Socket client = server.accept();
39                new Thread(() -> handleClient(client)).start();
40            }

```

String message = scanner.nextLine()
web-browser

45:1 CRLF UTF-8

```

40
41     } catch (IOException e) {
42         System.err.println("Server error: " + e.getMessage());
43     }
44 }
45
46 1 usage
47 @ private static void handleClient(Socket client) {
48     try (BufferedReader in = new BufferedReader(new InputStreamReader(client.getInputStream()))) {
49         String msg = in.readLine();
50         System.out.println("Received from peer: " + msg);
51     } catch (IOException e) {
52         System.err.println("Error reading from peer: " + e.getMessage());
53     }
54 }
55
56 1 usage
57 private static void sendMessageToPeer(String ip, int port, String message) {
58     try (Socket socket = new Socket(ip, port);
59         PrintWriter out = new PrintWriter(socket.getOutputStream(), autoFlush: true)) {
60         out.println(message);
61         System.out.println("Message sent to peer");
62     } catch (IOException e) {
63         System.err.println("Failed to connect to peer: " + e.getMessage());
64     }
65 }
66 }

```

Рис. 1 – Реалізація P2P

Ця програма створює просту P2P (peer-to-peer) модель, у якій кожен учасник (вузол) може:
отримувати повідомлення від інших (тобто працювати як сервер),
відправляти повідомлення іншому учаснику (тобто працювати як клієнт).

Як працює програма:

Коли запускається програма, вона одразу створює сервер, який слухає порт 12345. Це потрібно, щоб інші учасники могли до вас підключитися й надіслати повідомлення.

Потім програма питає користувача:

"Чи хочеш підключитися до іншого учасника?"

Якщо так, користувач вводить:

IP-адресу іншого учасника (наприклад, localhost для тесту),

Текст повідомлення.

Програма підключається до цього IP, надсилає повідомлення, і воно відображається у консольній програмі отримувача.

Приклад:

Ви запускаєте програму на двох комп'ютерах (або двічі на одному).

На одному вводите "ні" — він просто чекає на повідомлення.

На другому вводите "так", вказуєте IP першого, вводите текст.

Перший учасник отримає повідомлення й покаже його в консолі.

Ключові особливості:

Один і той самий код працює як сервер і як клієнт.

Простий спосіб обміну повідомленнями між будь-якими двома комп'ютерами в мережі.

Програма використовує TCP-з'єднання (тобто надійна доставка).

Асинхронність: Для обробки з'єднань на сервері використовується окремий потік, що дозволяє серверу продовжувати працювати незалежно від підключень.

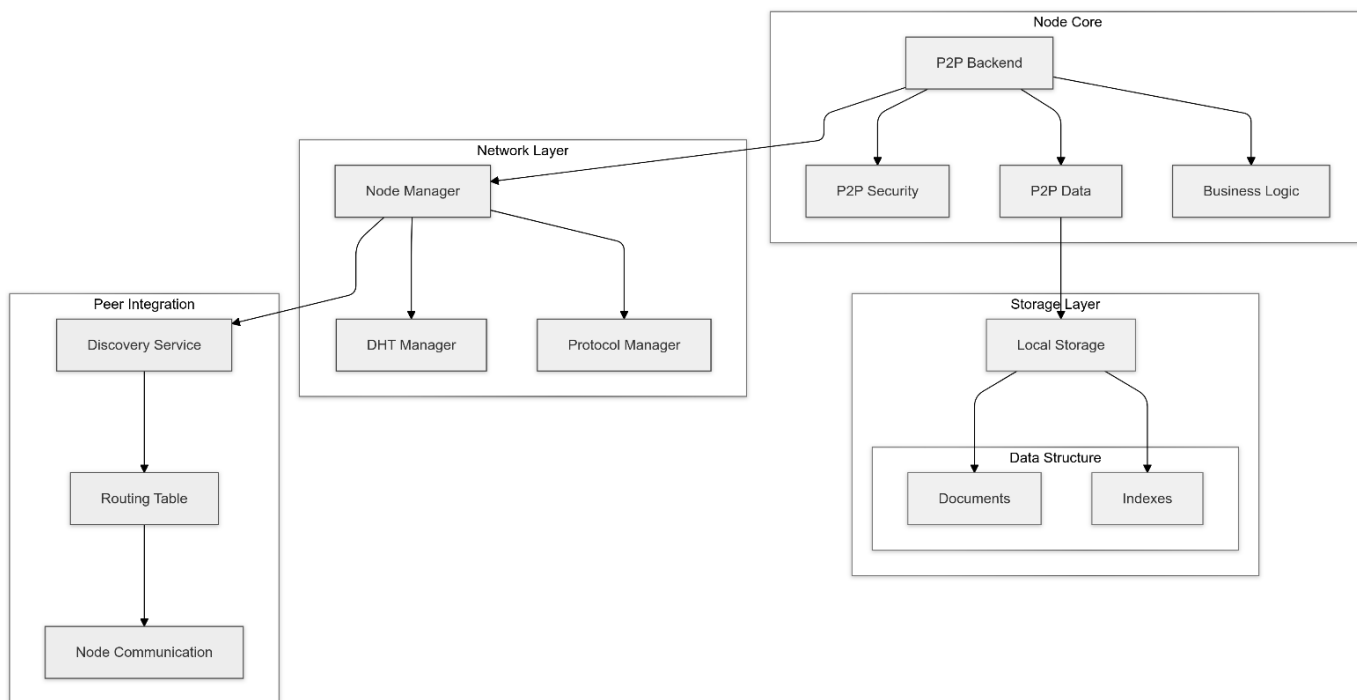


Рис. 2 – Структура P2P архітектури

1. Peer Integration (Інтеграція вузлів)

- **Discovery Service (Сервіс виявлення):** Відповідає за пошук нових вузлів у мережі.
- **Routing Table (Таблиця маршрутизації):** Зберігає інформацію про маршрути до інших вузлів.

- **Node Communication (Комунікація вузлів):** Забезпечує передачу даних між вузлами в мережі.

2. Network Layer (Мережевий шар)

- **Node Manager (Менеджер вузлів):** Керує взаємодією вузлів на рівні мережі.
 - **DHT Manager (Менеджер розподіленої хеш-таблиці):** Відповідає за

управління даними в розподіленій хеш-таблиці (DHT) для ефективного пошуку ресурсів.

- **Protocol Manager (Менеджер протоколів):** Забезпечує роботу різних протоколів для взаємодії вузлів.

3. Node Core (Ядро вузла)

- **P2P Backend (Бекенд P2P):** Центральний компонент, який забезпечує основну функціональність вузла.
 - **P2P Security (Безпека P2P):** Реалізує механізми аутентифікації, шифрування та захисту даних.
 - **P2P Data (Дані P2P):** Керує обробкою та передачею даних.
 - **Business Logic (Бізнес-логіка):** Реалізує логіку додатків і роботу з даними.

4. Storage Layer (Шар зберігання даних)

- **Local Storage (Локальне зберігання):** Відповідає за збереження даних на рівні окремого вузла.
 - **Data Structure (Структура даних):**
 - **Documents (Документи):** Зберігає файли чи дані у вигляді документів.
 - **Indexes (Індекси):** Забезпечує швидкий доступ до збережених даних.

Основна взаємодія компонентів:

1. **Peer Integration** з'єднує вузли між собою через **Discovery Service** і **Routing Table**.
2. **Network Layer** координує мережеву взаємодію між вузлами.
3. **Node Core** реалізує основну функціональність P2P-системи.
4. **Storage Layer** забезпечує локальне зберігання даних і оптимізацію доступу до них.

Посилання на репозиторій:

https://github.com/Elmir2/trpz_labs

Висновок: У ході виконання лабораторної роботи було розглянуто різні підходи до організації взаємодії додатків, а саме Client-Server, Peer-to-Peer та Service-Oriented Architecture. Детально вивчено їх принципи, переваги, недоліки та сфери застосування. Особливу увагу було приділено Peer-to-Peer архітектурі, яка демонструє високий рівень децентралізації, масштабованості та стійкості до відмов. У рамках практичної частини роботи було створено структуру архітектури P2P-системи, що охоплює всі основні шари: інтеграцію вузлів, мережевий шар, ядро вузла та шар зберігання даних. Описано функціональність кожного з компонентів системи, зокрема, сервісів виявлення, маршрутизації, забезпечення безпеки та зберігання даних.

Створення такої моделі дозволило зрозуміти принципи роботи децентралізованих систем і застосування шаблонів проектування для побудови гнучких та ефективних архітектур. Це може бути корисним у майбутній розробці складних програмних систем, де важливі автономність, стійкість та оптимізація ресурсів.